Guión de trabajo tutorizado

Sesión práctica 2

Gestión y visualización de datos masivos

Profesor Dr. Carlos D. Barranco González
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA - Sesión Práctica 2 - Actividad.pdf» (Campus Virtual UPO).

Objetivos

• Se capaz de resolver casos prácticos relacionados con la gestión y visualización de datos masivos.

• Emplear herramientas adecuadas para la gestión de los datos.

• Ser capaz de diseñar un proyecto básico de big data en las fases de extracción de datos y almacenamiento.

Actividades

A1. (120 minutos – 6.7 puntos) En la sesión presencial hemos visto la articulación de diversos servicios útiles para contar con una infraestructura escalable. Concretamente, Atlas de MongoDB [1], como Database as a Servicie y AWS [2] como proveedor IaaS.

El objetivo de esta actividad es que el estudiante realice los mismos pasos para conectar estos servicios y hacer la carga de datos desde AWS (u otro proveedor de su elección) a Atlas, cargando concretamente un dump de Wikipedia de su elección. Tenga en cuenta las limitaciones del servicio Atlas en lo que se refiere al espacio disponible para las cuentas gratuitas (cuyo uso es el recomendado), por lo que deberá elegir un dump de Wikipedia [3] adecuado. Puede obtener una orientación en función del número de documentos que contiene cada Wikipedia [4].

Para realizar la carga deberá emplear la herramienta de carga que creó en Talend en la sesión anterior, adaptándola según las indicaciones dadas en clase. Cargará ésta en AWS (o en el proveedor elegido) y la ejecutará desde allí, teniendo en cuenta las consideraciones de selección de la región (si es que es aplicable en función del proveedor) dadas en clase.

Tenga en cuenta que la propia grabación de la sesión puede ser una buena guía para permitirle realizar todo el proceso. Obviamente, deberá considerar que las direcciones de la instancia de cloud que elija, así como del cluster de Atlas, serán las suyas propias.

Como resultado de la actividad, el alumno documentará todo el proceso, explicitando las actividades que se realizan y el sentido de las mismas. Todo ello quedará ilustrado con capturas de pantalla que deberán ser perfectamente contextualizadas con la explicaciones textuales adecuadas.

A2. (60 minutos – 3.3 puntos) En esta actividad veremos un caso práctico de despliegue de un servicio de almacenamiento orientado a contener conjuntos de datos dentro del ámbito de Big Data. Para ello, contaremos con la explicación de Olga Moreno, estudiante de doctorado de la Universidad Pablo de Olavide y personal técnico contratado en un proyecto de investigación sobre la gestión de datos de sensores [5].

Resuma en no más de 100 palabras el contenido de la charla de Olga Moreno. Además, responda de forma explícita y por separado a cada una de las preguntas.

• ¿Cuál es el objetivo final de toda la actividad que se ve en el video?

• ¿Por qué es relevante para el trabajo en un proyecto de Big Data?

• ¿Qué tecnologías se mencionan en el vídeo y cuál es la utilidad que se les da?

Finalmente, haga una comparación de costes entre la propuesta técnica del video empleando GCP y la obtención de un servicio similar en Atlas. Para ello, considere una base de datos de 50GB que se

almacenará durante 3 meses, y que la arquitectura requerirá una distribución en tres servidores de shards.

Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- Los alumnos tienen hasta el segundo lunes, desde la impartición de la clase, para devolver el cuestionario.

Bibliografía

1. MongoDB Atlas https://www.mongodb.com/products/platform/atlas-database

2. Amazon Web Services https://aws.amazon.com/es/

3. Wikipedia Dumps https://dumps.wikimedia.org/

4. List of Wikipedias - All Wikipedias ordered by number of articles https://meta.wikimedia.org/wiki/List_of_Wikipedias#All_Wikipedias_ordered_by_number_of_articles

5. MongoDB en Google Cloud Platform empleando Kubernetes. Olga Moreno,. https://youtu.be/MLiI4X0auFI

RESOLUCIÓN

A1. Carga de un dump de Wikipedia desde la nube (Google Cloud) a MongoDB Atlas

Planteamiento. Se reproduce la arquitectura de clase con dos servicios gestionados: **MongoDB Atlas** como base de datos como servicio (DBaaS) y un proveedor IaaS donde se ejecuta la carga. El enunciado permite elegir proveedor: se usa **Google Cloud Platform (Compute Engine)** en lugar de AWS, porque es el proyecto de nube ya usado en el resto de actividades (BigQuery y Bigtable), con el crédito de la prueba gratuita.

Componente	En clase	En esta resolución	Por qué
DBaaS	MongoDB Atlas M0	MongoDB Atlas **M0 (gratuito)**, Cluster0, MongoDB 8.0, *replica set* de 3 nodos	Igual; 512 MB de almacenamiento
Región de Atlas	La de AWS elegida	**GCP / Belgium (europe-west1)**	Misma región que la VM y la más cercana a España con nivel gratuito
IaaS	EC2 (AWS)	**Compute Engine** `mbda-loader`, e2-small, Debian 12, zona europe-west1-b	Mismo proveedor y región que el clúster
Herramienta de carga	Job de Talend exportado (*Build Job*)	Script Python `etl_atlas.py` (misma lógica XPath → documento → inserción)	Talend no ejecutable en el equipo disponible (véase la A1 de la Sesión Práctica 1)
Datos	Dump de una Wikipedia pequeña	**Wikipedia en aragonés** (`anwiki-latest-pages-articles.xml.bz2`, 55 MB, dump del 01/09/2026, 220.156 páginas)	Cabe en 512 MB porque sólo se guardan metadatos, no el texto

1) Clúster en Atlas

Se crea el clúster gratuito M0 en Google Cloud, región Bélgica (`europe-west1`), sin datos de ejemplo. Elegir la región del clúster igual que la de la máquina de carga evita tráfico entre regiones: menos latencia en cada `insert_many` y sin coste de salida entre regiones.

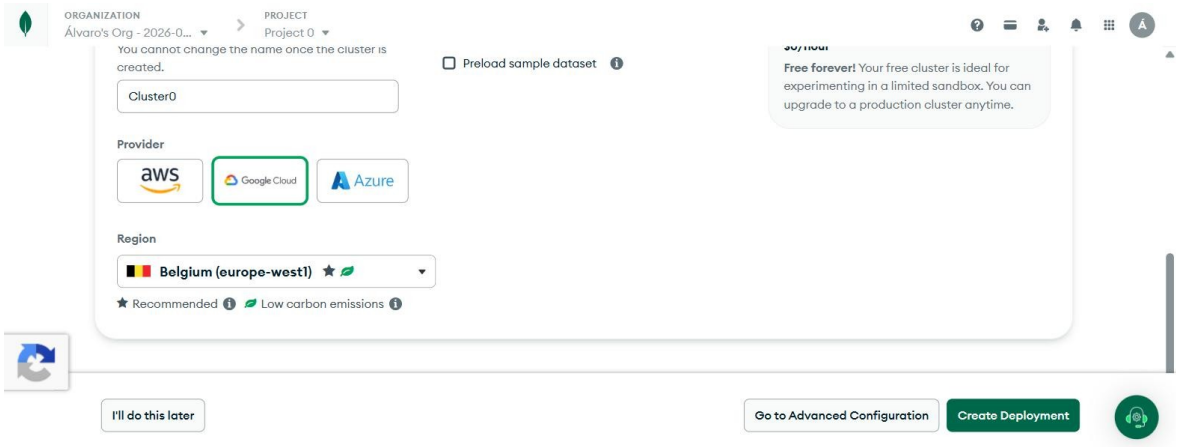


Figura 1. Configuración del clúster: nivel Free (M0), proveedor Google Cloud, región Belgium (europe-west1).

2) Máquina de carga en Compute Engine

```
gcloud compute instances create mbda-loader --zone=europe-west1-b \
  --machine-type=e2-small --image-family=debian-12 --image-project=debian-cloud --
boot-disk-size=10GB
```

```
gcloud compute ssh mbda-loader --zone=europe-west1-b
```

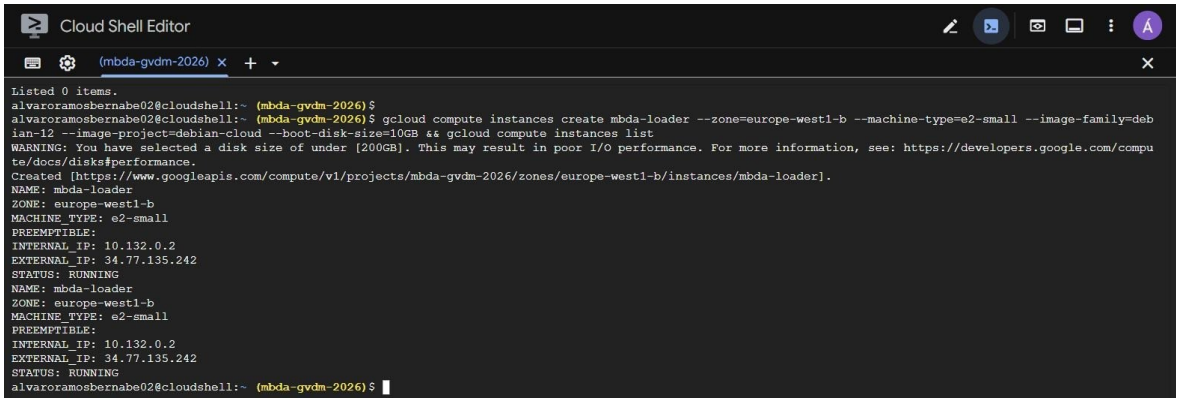


Figura 2. Creación de la VM desde Cloud Shell: IP externa 34.77.135.242, estado RUNNING.

En la VM se instala Python con un entorno virtual y el controlador `pymongo` (equivalente a las librerías que empaqueta el *Build Job* de Talend), y se descarga el dump **directamente desde dumps.wikimedia.org**. Así el fichero no pasa por el equipo local: la transferencia se hace dentro de la red del proveedor, que es la idea de ejecutar la carga en la nube.

```
sudo apt-get install -y python3-venv wget
python3 -m venv ~/v && ~/v/bin/pip install pymongo
wget https://dumps.wikimedia.org/anwiki/latest/anwiki-latest-pages-articles.xml.bz2
```

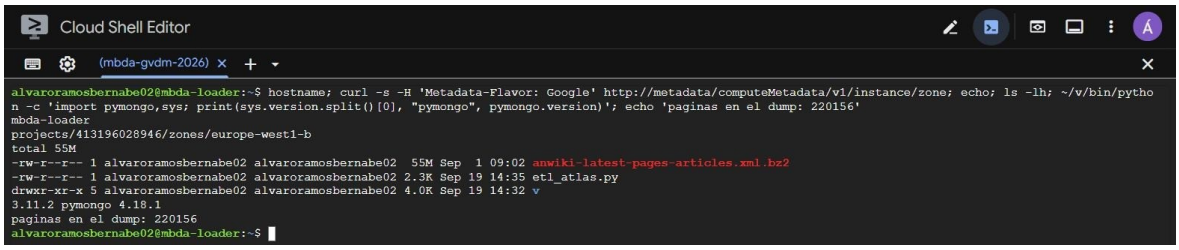


Figura 3. VM preparada: zona europe-west1-b, dump de 55 MB, script de carga, Python 3.11 y pymongo 4.18; el dump tiene 220.156 páginas.

3) Seguridad en Atlas: acceso de red y usuario

Atlas rechaza cualquier conexión que no venga de una IP autorizada. En *Network Access* se añade **sólo la IP de la VM** (`34.77.135.242/32`), no `0.0.0.0/0`. En *Database Access* se crea un usuario de base de datos (`mbda_loader`) con permiso de lectura y escritura. La contraseña no se escribe en el código: el script la pide por teclado sin mostrarla y la pasa por la variable de entorno `MONGO_URI`.

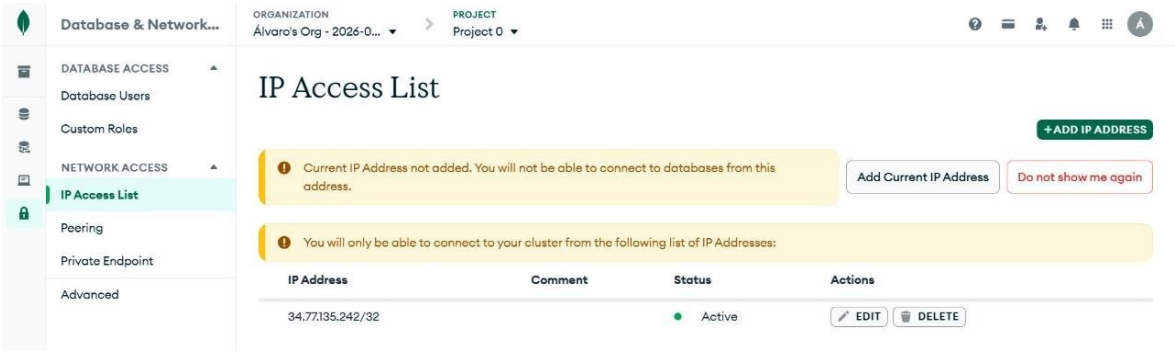


Figura 4. IP Access List: sólo la IP de la VM (34.77.135.242/32), activa.

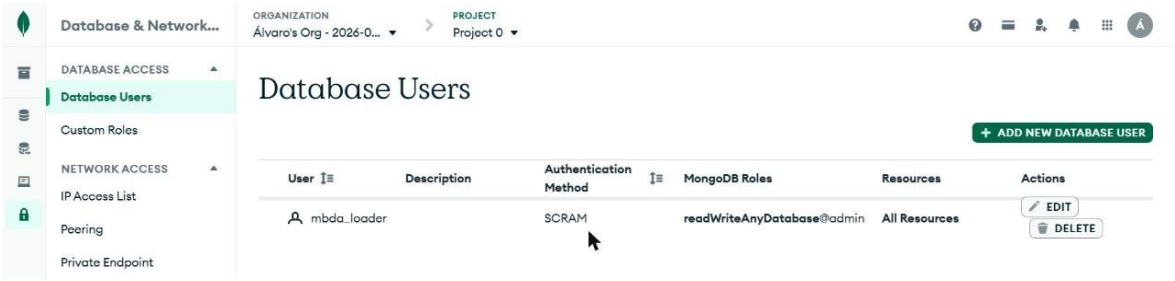


Figura 5. Usuario de base de datos mbda_loader (SCRAM, readWriteAnyDatabase).

4) Adaptación de la herramienta de carga

Respecto a la carga local de la Sesión Práctica 1, sólo cambia el destino, igual que en el componente `tMongoDBOutput` de Talend. Se pasa de `mongodb://localhost:27017` a la cadena **`mongodb+srv://`** de Atlas. Esa cadena resuelve por DNS los tres nodos del *replica set*, activa TLS y autentica contra `admin`. La extracción XPath de cada `<page>` y la inserción por lotes de 1.000 documentos son las mismas. Al final, el script comprueba el recuento, calcula el top 10 de contribuidores y consulta el tamaño ocupado.

```
#!/usr/bin/env python3
"""Carga de un dump de Wikipedia en MongoDB Atlas desde una VM de Compute Engine.
Misma lógica que el job de Talend de la Sesión Práctica 1 (XPath -> documento -> tMongoDBOutput),
cambiando sólo el destino: la cadena de conexión de Atlas (mongodb+srv, TLS) se lee de MONGO_URI
para no escribir la contraseña en el código."""
import bz2, os, sys, time, xml.etree.ElementTree as ET
from datetime import datetime
from pymongo import MongoClient

src = sys.argv[1]
NS = '{http://www.mediawiki.org/xml/export-0.11/}'
cli = MongoClient(os.environ['MONGO_URI'], appname='mbda-etl-gce')
col = cli['wikipedia']['articles']
col.drop()
t0 = time.time(); lote, total = [], 0
for ev, el in ET.iterparse(bz2.open(src), events=('end',)):
    if el.tag != NS + 'page':
        continue
    rev = el.find(NS + 'revision'); con = rev.find(NS + 'contributor'); txt = rev.find(NS +
'text')
    lote.append({
        'title': el.findtext(NS + 'title'),
        'ns': int(el.findtext(NS + 'ns')),
        'id': int(el.findtext(NS + 'id')),
        'redirect': el.find(NS + 'redirect') is not None,
        'revisionid': int(rev.findtext(NS + 'id')),
        'timestamp': datetime.fromisoformat(rev.findtext(NS + 'timestamp').replace('Z',
'+00:00')),
```

```
        'contributorusername': con.findtext(NS + 'username') or con.findtext(NS + 'ip'),
        'contributorid': int(con.findtext(NS + 'id')) if con.findtext(NS + 'id') else None,
        'textbytes': int(txt.get('bytes', 0)) if txt is not None else 0,
    })
    total += 1
    if len(lote) == 1000:
        col.insert_many(lote); lote = []
        if total % 20000 == 0: print(f'  {total} documentos...', flush=True)
    el.clear()
if lote: col.insert_many(lote)
print(f'Documentos cargados en wikipedia.articles: {total} en {time.time() - t0:.0f} s')
print('count_documents:', col.count_documents({}))
print('Top 10 contribuidores (último editor):')
for d in col.aggregate([{'$group': {'_id': '$contributorusername', 'count': {'$sum': 1}}},
                        {'$sort': {'count': -1}}, {'$limit': 10}]):
    print(f"  {d['count']:>6}  {d['_id']}")
st = cli['wikipedia'].command('dbStats')
print(f"dataSize={st['dataSize']/1e6:.1f} MB  storageSize={st['storageSize']/1e6:.1f} MB
indexSize={st['indexSize']/1e6:.1f} MB")
```

```
#!/bin/bash
# Pide la contraseña del usuario de Atlas sin mostrarla y lanza la carga
read -s -p 'Contraseña de mbda_loader: ' P; echo
P=$(python3 -c 'import urllib.parse,sys; print(urllib.parse.quote(sys.argv[1], safe=""))'
"$P")
export MONGO_URI="mongodb+srv://mbda_loader:$P@cluster0.c9zxos4.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0"
time ~/v/bin/python etl_atlas.py anwiki-latest-pages-articles.xml.bz2
```

5) Ejecución en la nube y verificación

Se lanza `./cargar.sh` en la VM: pide la contraseña, arma la cadena `mongodb+srv` y ejecuta la carga contra Atlas.

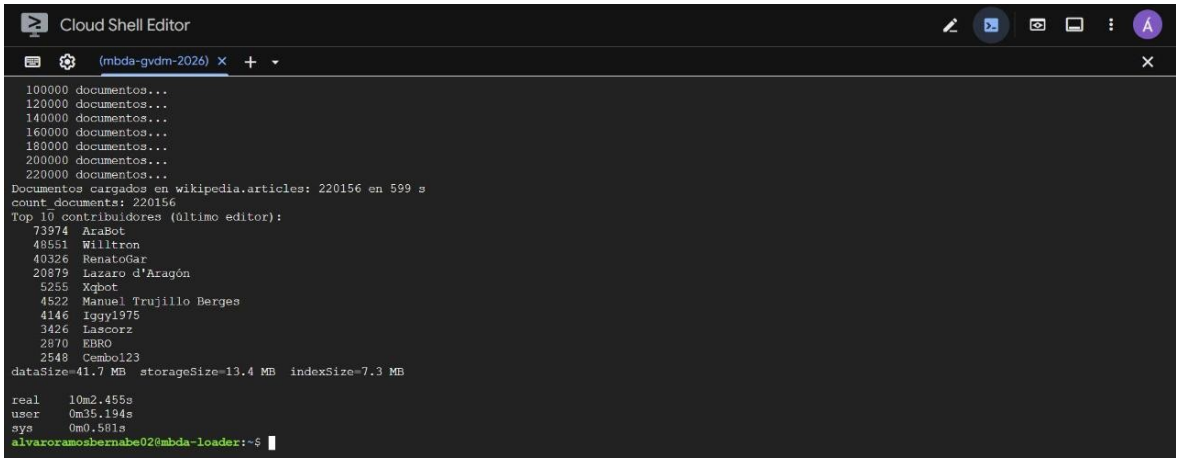


Figura 6. Ejecución de la carga en la VM: 220.156 documentos en 599 s, recuento, top 10 de contribuidores y tamaño en Atlas.

Resultado	Valor
Documentos cargados (`wikipedia.articles`)	220.156 (igual que las páginas del dump)
Tiempo de carga	599 s (≈ 10 min, ≈ 370 documentos/s)
Tamaño de los datos (`dataSize`)	41,7 MB (189 B de media por documento)
Almacenamiento en disco (`storageSize`, comprimido)	13,4 MB + 7,3 MB de índice `_id`: ≈ 4 % de los 512 MB del M0

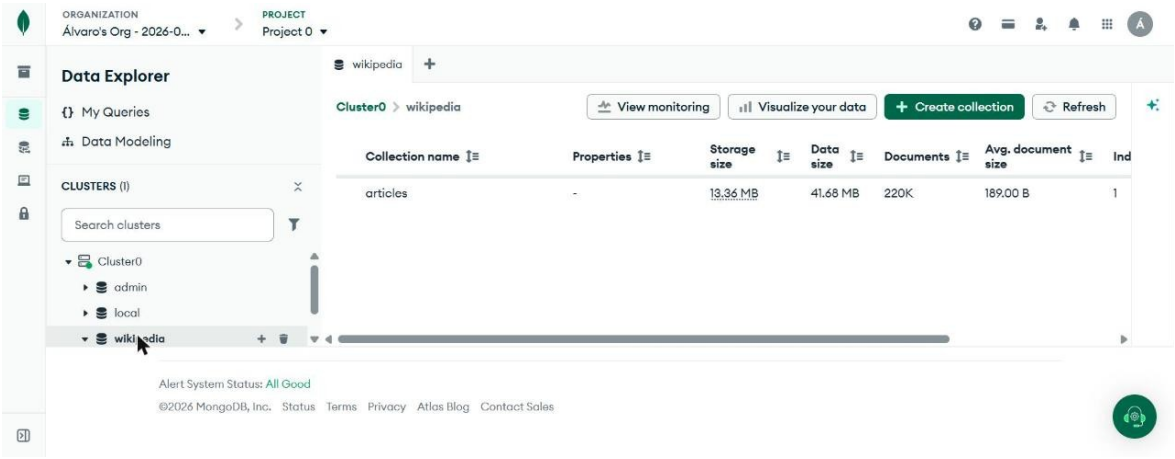


Figura 7. Data Explorer de Atlas: base wikipedia, colección articles con 220K documentos, 13,36 MB en disco.

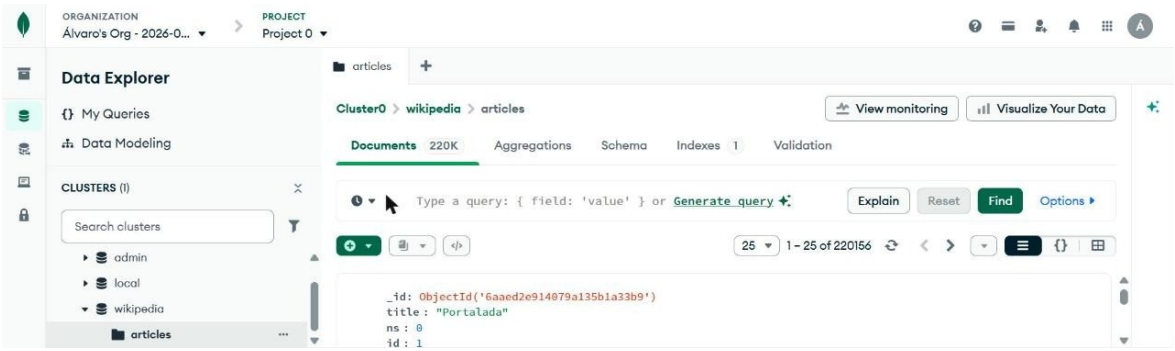


Figura 8. Documentos de la colección (1-25 de 220.156): la carga se ve desde la consola de Atlas.

Top 10 de contribuidores (último editor de cada página): AraBot (73.974), Willtron (48.551), RenatoGar (40.326), Lazaro d'Aragón (20.879), Xqbot (5.255), Manuel Trujillo Berges (4.522), Iggy1975 (4.146), Lascorz (3.426), EBRO (2.870) y Cembo123 (2.548). Igual que en la Wikipedia extremeña, el primero es un **bot** (AraBot, un tercio de las páginas) y hay otro más (Xqbot). Para medir la actividad humana habría que filtrarlos.

Observaciones. (1) El cuello de botella es el nivel gratuito: el M0 comparte recursos y limita las operaciones por segundo, y la carga de ≈ 370 documentos/s está muy por debajo de lo que admite la VM (el parseo usa 35 s de CPU de 600). Con un nivel dedicado o con `insert_many` desordenado y más grande sería más rápida. (2) Guardar sólo los metadatos permite cargar una Wikipedia de 220.000 páginas en menos del 5 % del espacio. Con el texto completo (el dump descomprimido ocupa varios cientos de MB) no cabría en el M0: esa es la razón de elegir el dump y los campos con cuidado, como advierte el enunciado.

6) Limpieza

Terminada la carga, se **elimina la VM** para no consumir crédito. El clúster M0 es gratuito y se puede conservar. Conviene borrar el usuario `mbda_loader` y la entrada de IP, porque esa IP puede reasignarse a otra máquina de Google Cloud.

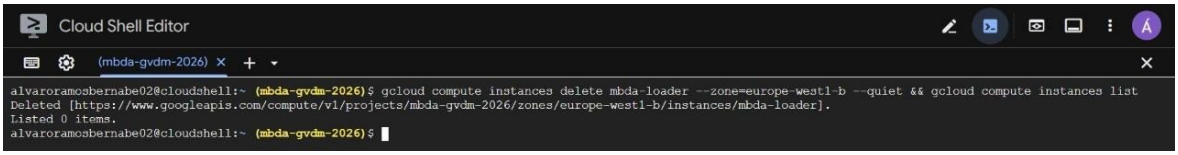


Figura 9. Borrado de la VM: `gcloud compute instances list` ya no devuelve instancias.

A2. Despliegue de MongoDB en Google Cloud Platform con Kubernetes (charla de Olga Moreno)

La charla (vídeo «MongoDB en Google Cloud Platform con Kubernetes», 40 min, canal de Carlos D. Barranco, 2019) se ha analizado a partir de su transcripción, que se adjunta en la carpeta Materiales.

Resumen (≤ 100 palabras)

Olga Moreno explica cómo desplegar MongoDB en Google Cloud Platform con Kubernetes. Repasa primero los conceptos de MongoDB: documentos BSON sin esquema fijo, *replica sets* (primario y secundarios) para la alta disponibilidad y clúster fragmentado con *shards*, servidores de configuración y *routers* `mongos` para escalar horizontalmente. Después presenta Kubernetes: clúster, nodos, *pods*, controladores de réplicas, *StatefulSets*, servicios y volúmenes. Por último, con los ficheros YAML y los scripts de su repositorio, crea en GKE un clúster con configuración replicada, tres *shards* y dos *routers*. Inserta y consulta datos para comprobarlo y destruye la infraestructura para no generar costes.

(98 palabras)

¿Cuál es el objetivo final de toda la actividad que se ve en el vídeo?

Tener un **servicio de almacenamiento MongoDB escalable y de alta disponibilidad desplegado de forma automática y reproducible** en la nube. Es un clúster fragmentado sobre Kubernetes en GKE que se crea y destruye con scripts y ficheros de definición (infraestructura como código), para alojar los conjuntos de datos de un proyecto de investigación sobre datos de sensores.

¿Por qué es relevante para el trabajo en un proyecto de Big Data?

- Resuelve la fase de **almacenamiento** del ciclo *adquisición → almacenamiento → análisis* visto en clase, que es la base de todo lo demás.
- Atiende a las «V»: el **sharding** reparte los datos y la carga entre nodos (**volumen**, **velocidad**) y el modelo de documentos admite datos heterogéneos (**variedad**).
- Las **replica sets** aportan redundancia y conmutación automática: un fallo de nodo no detiene el proyecto.
- Con **Kubernetes y la nube** se levantan o amplían nodos bajo demanda, se paga sólo por uso y el despliegue se repite sin errores manuales. Destruir la infraestructura al terminar es también una forma de controlar el coste.

¿Qué tecnologías se mencionan y qué utilidad se les da?

Tecnología	Utilidad en el vídeo
MongoDB (`mongod`)	Sistema gestor NoSQL orientado a documentos (BSON) en el que se almacenan los datos.
Replica set	Grupo de 3 instancias (1 primario y 2 secundarios) que da redundancia y alta disponibilidad, con elección automática de nuevo primario. Se usa en cada *shard* y en los servidores de configuración.
Sharding: *shards*, *chunks*, *shard key*	Particionado horizontal de colecciones en trozos (*chunks*) repartidos entre 3 *shards* según una clave, para escalar.
Servidores de configuración	*Replica set* con los metadatos del clúster: qué *chunk* está en qué *shard*.

`mongos` (router)	Punto de entrada de los clientes. Se despliegan 2 por disponibilidad y enrutan las consultas al *shard* adecuado.
Docker	Imagen oficial de MongoDB con la que se crean los contenedores.
Kubernetes	Orquestación de contenedores (creada por Google en 2014). Usa *pods* (uno por contenedor MongoDB), *StatefulSets* (nombres estables y disco propio por *pod*), *Services* (red y puerto 27017), volúmenes persistentes y *Secrets* (clave interna de autenticación). También un demonio en cada nodo para desactivar los *Transparent Huge Pages*, como recomienda MongoDB.
Google Cloud Platform – GKE y Compute Engine	Clúster Kubernetes gestionado, máquinas virtuales de los nodos y discos persistentes (8 GB por instancia en la demo). Requiere proyecto con facturación y las API habilitadas.
`gcloud` y `kubectl`	Herramientas de línea de comandos para autenticarse, crear y borrar el clúster y aplicar los ficheros YAML.
GitHub	Repositorio con la guía, los YAML de cada topología (*standalone*, *replica set*, *sharded*) y los scripts de creación y destrucción.

Comparación de costes: propuesta en GCP frente a MongoDB Atlas

Supuestos comunes: base de datos de **50 GB**, almacenada **3 meses** (≈ 2.190 horas), **3 shards**, cada uno con *replica set* de 3 miembros (factor de replicación 3). Precios de lista en USD, región `us-central1` o equivalente, sin impuestos ni descuentos. Cada *shard* guarda ≈ 17 GB y cada réplica una copia completa de su *shard*.

Opción 1 – Arquitectura del vídeo en GKE (autogestionada): 9 `mongod` de datos, 3 servidores de configuración y 2 `mongos`, es decir, 14 *pods*.

Concepto	Dimensionado	Precio unitario	Coste 3 meses
Nodos del clúster	3 × `e2-standard-4` (4 vCPU, 16 GB), suficientes para 14 *pods* pequeños	≈ 97,84 USD/mes por nodo	≈ 880 USD
Discos persistentes SSD	9 × 25 GB (datos) + 3 × 10 GB (configuración) = 255 GB	≈ 0,17 USD/GB·mes (pd-ssd)	≈ 130 USD
Tarifa de gestión de GKE	1 clúster zonal: 0,10 USD/h, cubierta por el crédito gratuito de 74,40 USD/mes	0 USD neto	0 USD
Total aproximado			**≈ 1.010 USD (≈ 337 USD/mes)**

Opción 2 – MongoDB Atlas (servicio gestionado, DBaaS): los clústeres fragmentados exigen nivel **M30 o superior**. Atlas factura cada *shard* como un clúster de su nivel (3 nodos de datos) y añade los servidores de configuración a una tarifa aparte.

Concepto	Dimensionado	Precio unitario	Coste 3 meses
Shards	3 × M30 (8 GB RAM, 2 vCPU, 40 GB incluidos por *shard*, suficientes para ≈ 17 GB)	0,54 USD/h por shard (388,80 USD/mes)	≈ 3.550 USD
Servidores de configuración	3 nodos gestionados por Atlas	Tarifa aparte (no publicada en la tabla general)	+ importe adicional
Almacenamiento	Incluido (40 GB por *shard*)	—	0 USD
Total aproximado			**≈ 3.550 USD + config. (≈ 1.170 USD/mes)**

Conclusión. La propuesta en GKE es aproximadamente **3,5 veces más barata** en infraestructura: unos 1.000 USD frente a más de 3.500 USD en 3 meses. Pero no incluye el **coste de operación**, que en Atlas va incluido: parches y actualizaciones, copias de seguridad, monitorización y alertas, escalado automático, seguridad (TLS, cifrado, control de acceso) y SLA de disponibilidad. En GKE todo eso lo asume el equipo, que además necesita conocimientos de Kubernetes. Para un **proyecto de investigación con personal técnico propio y cargas temporales** (se levanta y se destruye), como el del vídeo, la opción GCP/Kubernetes es razonable y más económica. Para un **servicio en producción** con requisitos de disponibilidad y sin equipo de operaciones, el sobrecoste de Atlas suele compensar. Las cifras son estimaciones con precios de lista y deben confirmarse con las calculadoras oficiales (Google Cloud Pricing Calculator y Atlas) para la región real, p. ej. Madrid, `europe-southwest1`, que suele ser algo más cara que `us-central1`.

Referencias

[1] O. Moreno, «MongoDB en Google Cloud Platform con Kubernetes», vídeo, 2019. https://youtu.be/MLiI4X0auFI

[2] MongoDB Atlas – Pricing. https://www.mongodb.com/pricing

[3] MongoDB Atlas – Cluster Configuration Costs. https://www.mongodb.com/docs/atlas/billing/cluster-configuration-costs/

[4] Google Kubernetes Engine pricing. https://cloud.google.com/kubernetes-engine/pricing

[5] Precios de e2-standard-4 (us-central1). https://www.economize.cloud/resources/gcp/pricing/compute-engine/e2-standard-4/